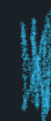




Diffusion articles



Вопросы



Предложили
в статье

Improved DDPM

Improved Denoising Diffusion Probabilistic Models

Alex Nichol^{*1} Prafulla Dhariwal^{*1}

Abstract

Denoising diffusion probabilistic models (DDPM) are a class of generative models which have recently been shown to produce excellent samples. We show that with a few simple modifications, DDPMs can also achieve competitive log-likelihoods while maintaining high sample quality. Additionally, we find that learning variances of the reverse diffusion process allows sampling with an order of magnitude fewer forward passes with a negligible difference in sample quality, which is important for the practical deployment of these models. We additionally use precision and re-

(2020) showed extremely good results on the CIFAR-10 (Krizhevsky, 2009) and LSUN (Yu et al., 2015) datasets, it is unclear how well DDPMs scale to datasets with higher diversity such as ImageNet. Finally, while Chen et al. (2020b) found that DDPMs can efficiently generate audio using a small number of sampling steps, it has yet to be shown that the same is true for images.

In this paper, we show that DDPMs can achieve log-likelihoods competitive with other likelihood-based models, even on high-diversity datasets like ImageNet. To more tightly optimise the variational lower-bound (VLB), we learn the reverse process variances using a simple reparameterization and a hybrid learning objective that combines the

Improved DDPM

Зашумление

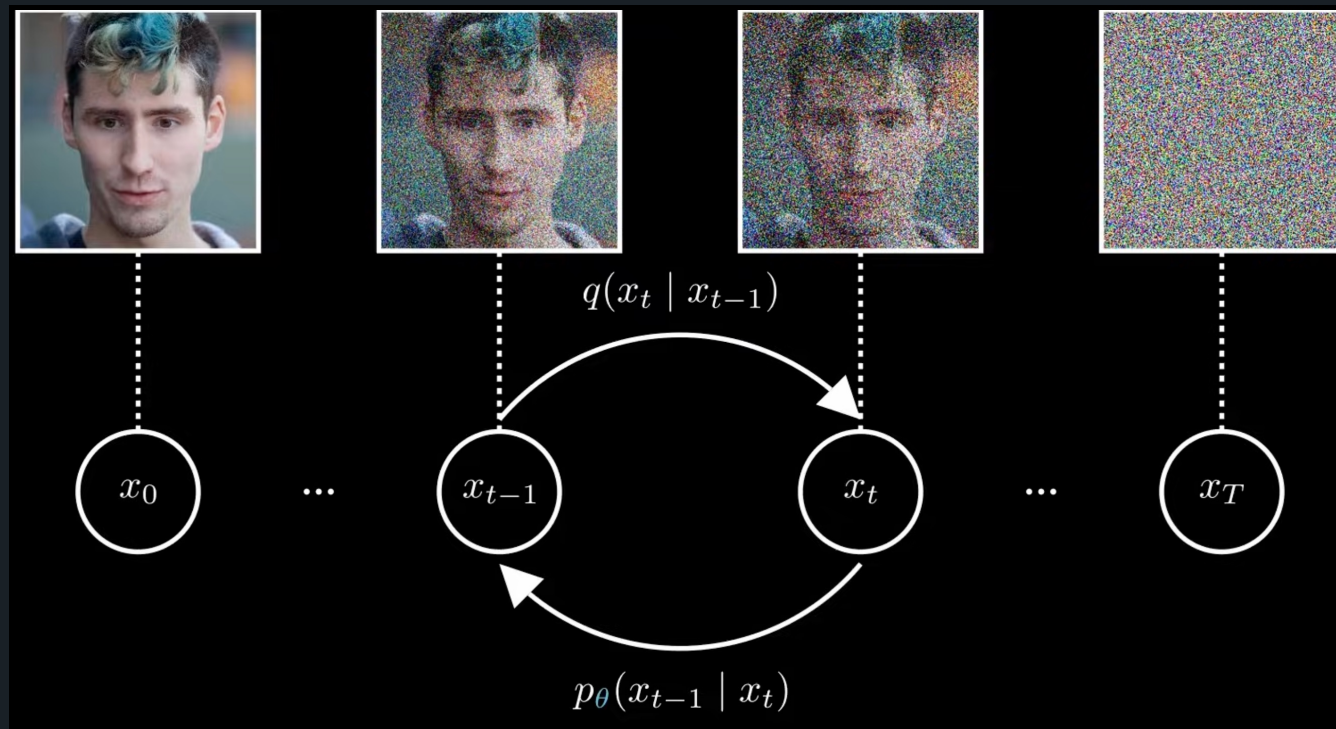
$$q(x_t|x_{t-1}) := \mathcal{N}(x_t; \sqrt{1 - \beta_t}x_{t-1}, \beta_t\mathbf{I}) \quad (2)$$

Расшумление

$$p_\theta(x_{t-1}|x_t) := \mathcal{N}(x_{t-1}; \mu_\theta(x_t, t), \Sigma_\theta(x_t, t)) \quad (3)$$

Одношаговое зашумление

$$q(x_t|x_0) = \mathcal{N}(x_t; \sqrt{\bar{\alpha}_t}x_0, (1 - \bar{\alpha}_t)\mathbf{I}) \quad (8)$$



Improved DDPM – можно учить по разному

vlb loss – в лоб

$$L_{\text{vlb}} := L_0 + L_1 + \dots + L_{T-1} + L_T \quad (4)$$

$$L_0 := -\log p_\theta(x_0|x_1) \quad (5)$$

$$L_{t-1} := D_{KL}(q(x_{t-1}|x_t, x_0) \parallel p_\theta(x_{t-1}|x_t)) \quad (6)$$

$$L_T := D_{KL}(q(x_T|x_0) \parallel p(x_T)) \quad (7)$$

$$\nabla_{\Sigma_\theta} L_{\text{vlb}} = ??$$

$$\nabla_{\mu_\theta} L_{\text{vlb}} = ??$$

simple loss – картинки лучше чем у vlb

$$L_{\text{simple}} = E_{t, x_0, \epsilon} [||\epsilon - \epsilon_\theta(x_t, t)||^2]$$

$$\nabla_{\Sigma_\theta} L_{\text{simple}} := 0 \quad \sigma_t^2 = \beta_t \text{ or } \sigma_t^2 = \tilde{\beta}_t \quad \text{дисперсию шедулим}$$

$$\nabla_{\mu_\theta} L_{\text{simple}} = ??$$

езде плохой log-likelihood

Не понял, почему обучение дисперсии в vlb портит обучение

Improved DDPM – улучшаем loglikelihood

1

Hybrid loss

$$L_{\text{hybrid}} = L_{\text{simple}} + \lambda L_{\text{vlb}}$$

$$\nabla_{\Sigma_{\theta}} L_{\text{simple}} := 0 \quad \nabla_{\Sigma_{\theta}} L_{\text{vlb}} = ??$$

$$\nabla_{\mu_{\theta}} L_{\text{simple}} = ?? \quad \nabla_{\mu_{\theta}} L_{\text{vlb}} \stackrel{*}{=} 0$$

2

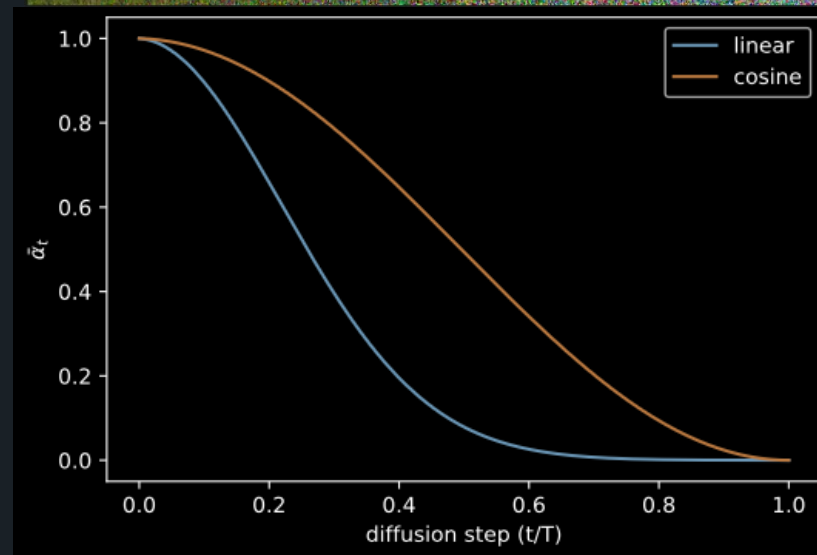
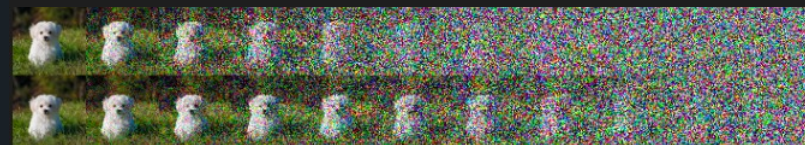
Дисперсию учим, но не в лоб

$$\Sigma_{\theta}(x_t, t) = \exp(v \log \beta_t + (1 - v) \log \tilde{\beta}_t)$$

3

Косинусный шедюлер
дисперсии forward path

$$\bar{\alpha}_t = \frac{f(t)}{f(0)}, \quad f(t) = \cos \left(\frac{t/T + s}{1 + s} \cdot \frac{\pi}{2} \right)^2$$



Improved DDPM – улучшаем loglikelihood

4 vlb loss (resampletd) – в лоб

$$L_{\text{vlb}} := L_0 + L_1 + \dots + L_{T-1} + L_T \quad (4)$$

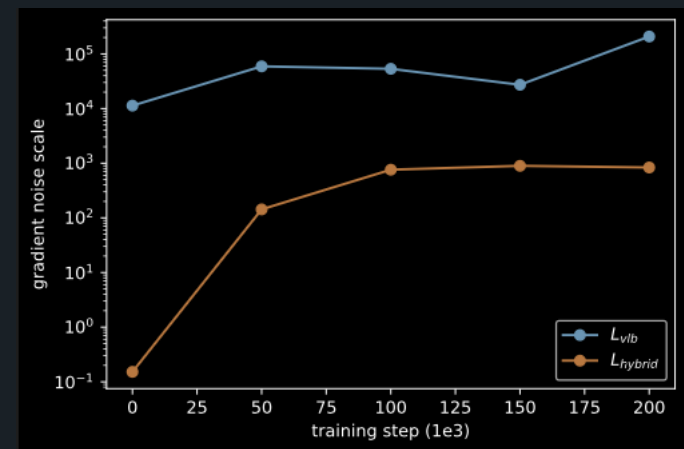
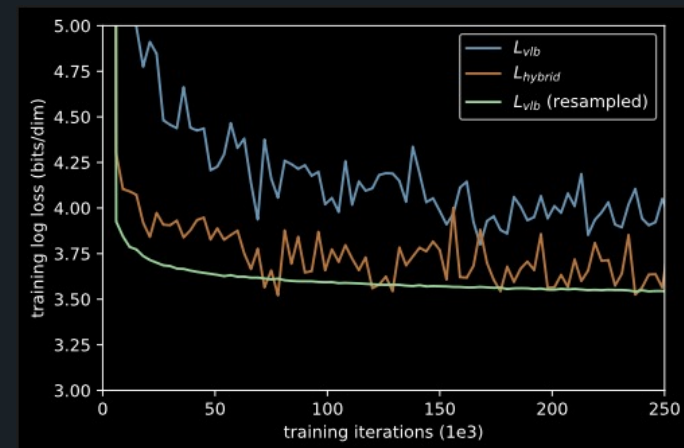
$$L_0 := -\log p_\theta(x_0|x_1) \quad (5)$$

$$L_{t-1} := D_{KL}(q(x_{t-1}|x_t, x_0) \parallel p_\theta(x_{t-1}|x_t)) \quad (6)$$

$$L_T := D_{KL}(q(x_T|x_0) \parallel p(x_T)) \quad (7)$$

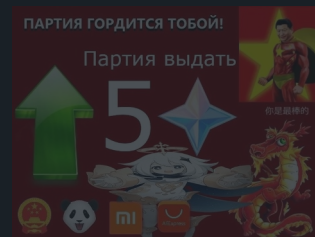
$$L_{\text{vlb}} = E_{t \sim p_t} \left[\frac{L_t}{p_t} \right], \text{ where } p_t \propto \sqrt{E[L_t^2]} \text{ and } \sum p_t = 1$$

Не понял, почему это помогло
Не понимаю как мы формируем батчи?



Improved DDPM – улучшаем loglikelihood

Model	ImageNet	CIFAR
Glow (Kingma & Dhariwal, 2018)	3.81	3.35
Flow++ (Ho et al., 2019)	3.69	3.08
PixelCNN (van den Oord et al., 2016c)	3.57	3.14
SPN (Menick & Kalchbrenner, 2018)	3.52	-
NVAE (Vahdat & Kautz, 2020)	-	2.91
Very Deep VAE (Child, 2020)	3.52	2.87
PixelSNAIL (Chen et al., 2018)	3.52	2.85
Image Transformer (Parmar et al., 2018)	3.48	2.90
Sparse Transformer (Child et al., 2019)	3.44	2.80
Routing Transformer (Roy et al., 2020)	3.43	-
DDPM (Ho et al., 2020)	3.77	3.70
DDPM (cont flow) (Song et al., 2020b)	-	2.99
Improved DDPM (ours)	3.53	2.94

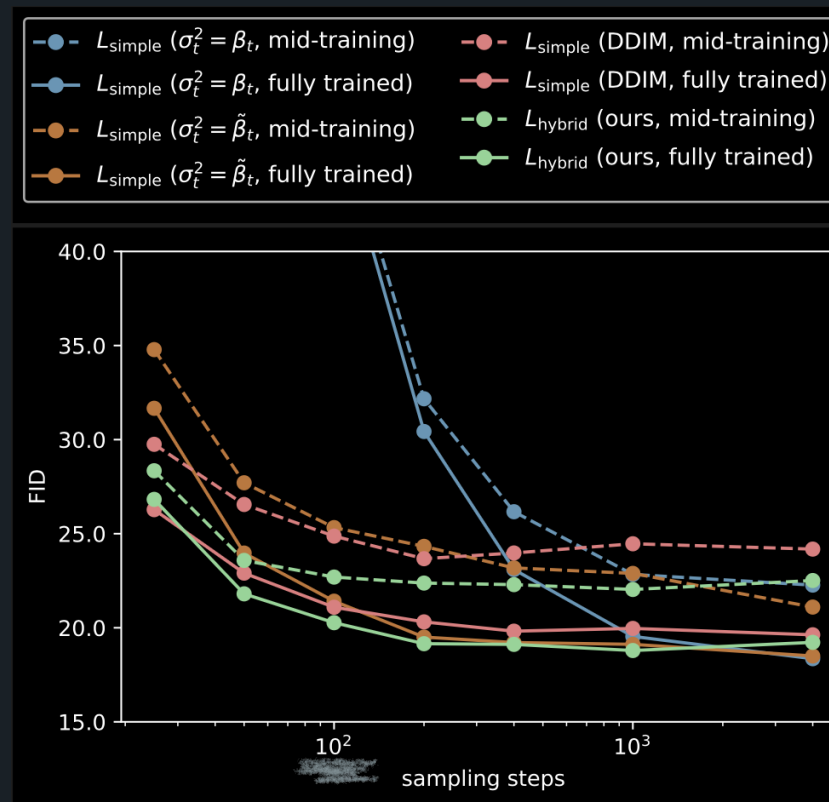


Improved DDPM – denoise за <<4000 steps

Steps → `S = np.linspace(1, T, K, dtype=int)`

Rescaling для нового множества шагов

$$\beta_{S_t} = 1 - \frac{\bar{\alpha}_{S_t}}{\bar{\alpha}_{S_{t-1}}}, \quad \tilde{\beta}_{S_t} = \frac{1 - \bar{\alpha}_{S_{t-1}}}{1 - \bar{\alpha}_{S_t}} \beta_{S_t}$$



На каком количестве шагов денойза учим в итоге? 4000?

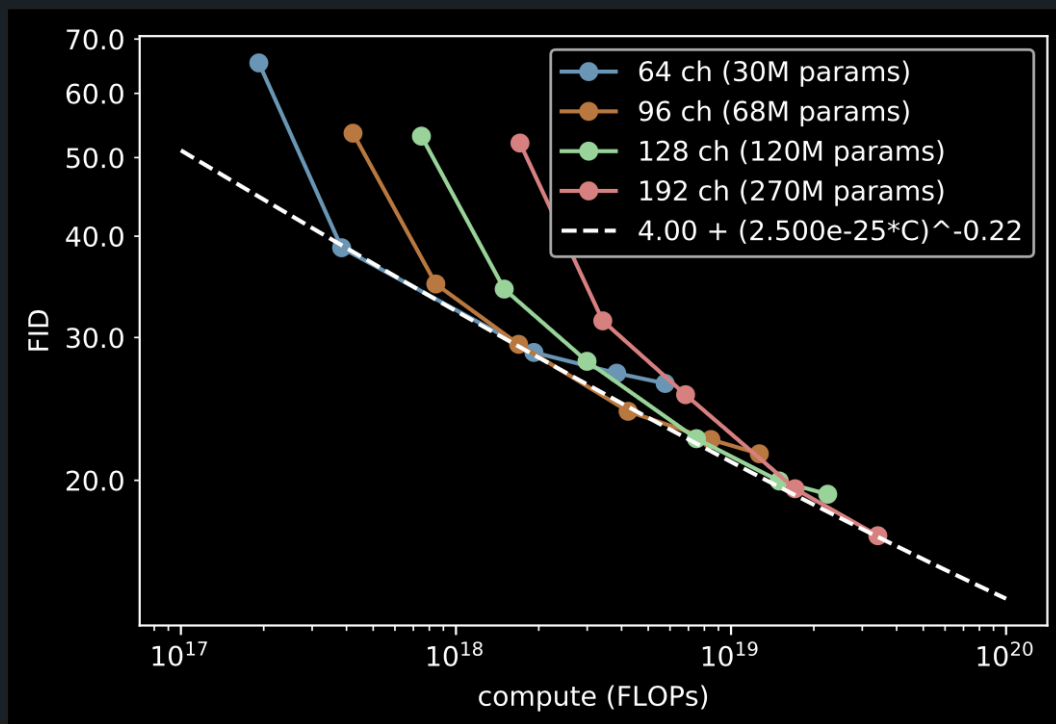
Improved DDPM vs GANs

Model	FID	Prec.	Recall
BigGAN-deep (Brock et al., 2018)	4.06	0.86	0.59
Improved Diffusion (small)	6.92	0.77	0.72
Improved Diffusion (large)	2.92	0.82	0.71

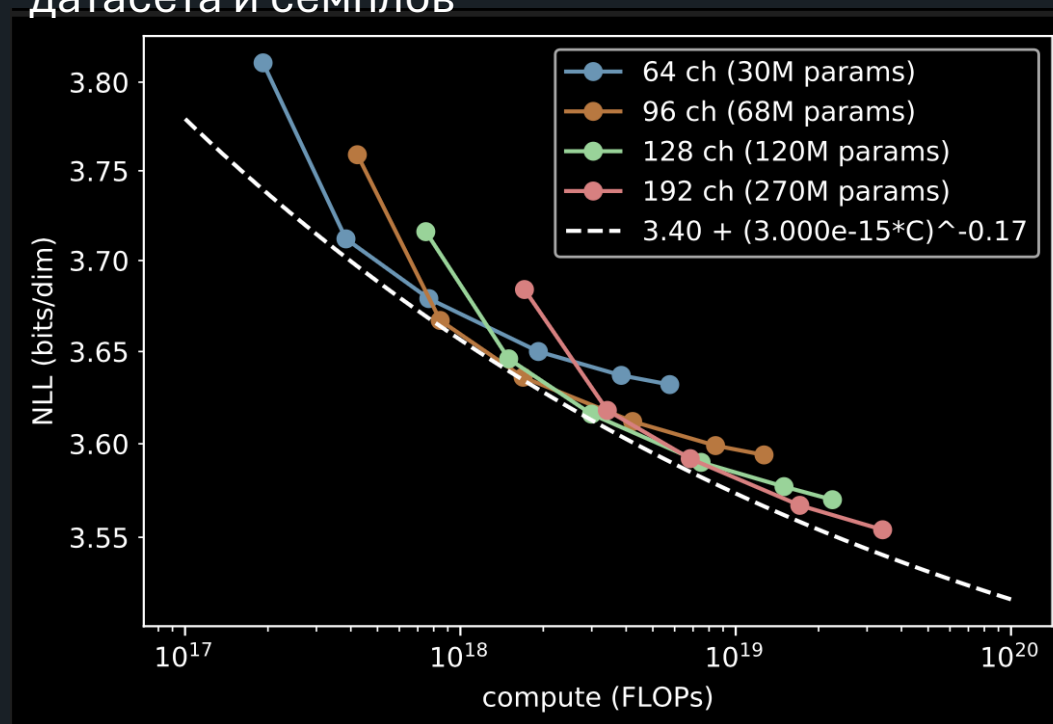


Improved DDPM – Scaling

FID - Реалистичность



NLL - Соответствие распределений датасета и семплов



Плохо понимаю FID, NLL. Я правильно их интерпретировал?

Score-Based Generative Modeling Through SDE

SCORE-BASED GENERATIVE MODELING THROUGH STOCHASTIC DIFFERENTIAL EQUATIONS

Yang Song*

Stanford University

yangsong@cs.stanford.edu

Jascha Sohl-Dickstein

Google Brain

jaschasd@google.com

Diederik P. Kingma

Google Brain

durk@google.com

Abhishek Kumar

Google Brain

abhishk@google.com

Stefano Ermon

Stanford University

ermon@cs.stanford.edu

Ben Poole

Google Brain

pooleb@google.com

ABSTRACT

Creating noise from data is easy; creating data from noise is generative modeling. We present a stochastic differential equation (SDE) that smoothly transforms a complex data distribution to a known prior distribution by slowly injecting noise, and a corresponding reverse-time SDE that transforms the prior distribution back into the data distribution by slowly removing the noise. Crucially, the reverse-time SDE depends only on the time-dependent gradient field (a.k.a., score) of the perturbed data distribution. By leveraging advances in score-based generative modeling, we can accurately estimate these scores with neural networks, and use numerical SDE solvers to generate samples. We show that this framework encapsulates previous

Score-Based Through SDE – что было до

SMLD

$$\mathbf{x}_i^m = \mathbf{x}_i^{m-1} + \epsilon_i \mathbf{s}_{\theta^*}(\mathbf{x}_i^{m-1}, \sigma_i) + \sqrt{2\epsilon_i} \mathbf{z}_i^m, \quad m = 1, 2, \dots, M,$$

```
1:  $\mathbf{x}_N \sim \mathcal{N}(\mathbf{0}, \sigma_{\max}^2 \mathbf{I})$ 
2: for  $i = N - 1$  to  $0$  do
3:   for  $j = 1$  to  $M$  do
4:      $\mathbf{z} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ 
5:      $\mathbf{x}_i \leftarrow \mathbf{x}_i + \epsilon_i \mathbf{s}_{\theta^*}(\mathbf{x}_i, \sigma_i) + \sqrt{2\epsilon_i} \mathbf{z}$ 
6: return  $\mathbf{x}_0$ 
```

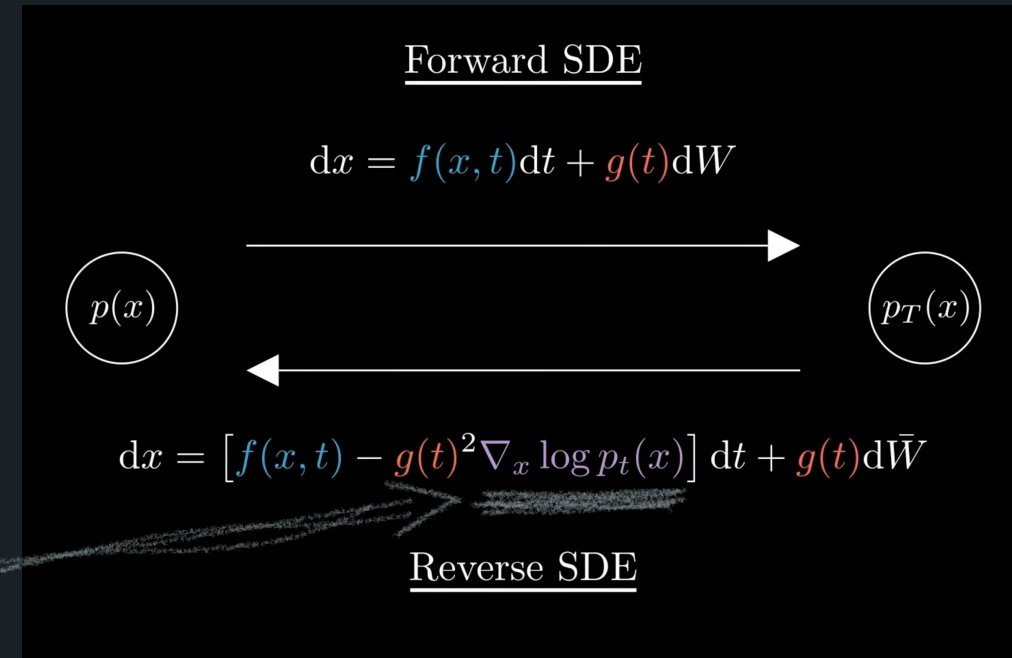
DDPM

```
1:  $\mathbf{x}_N \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ 
2: for  $i = N - 1$  to  $0$  do
3:    $\mathbf{x}'_i \leftarrow (2 - \sqrt{1 - \beta_{i+1}}) \mathbf{x}_{i+1} + \beta_{i+1} \mathbf{s}_{\theta^*}(\mathbf{x}_{i+1}, i + 1)$ 
4:    $\mathbf{z} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ 
5:    $\mathbf{x}_i \leftarrow \mathbf{x}'_i + \sqrt{\beta_{i+1}} \mathbf{z}$ 
```

Predictor

Score-Based Through SDE – идея

$$\begin{array}{ll}
 x_t, t \in \overline{1, T} & x(t), t \in [0, T] \\
 x_0 \sim p_{\text{data}} & x(0) \sim p_0 \Leftrightarrow p_{\text{data}} \\
 x_T \sim N(0, I) & x(T) \sim p_T - \text{от куда удобно семплить}
 \end{array}$$



$$\theta^* = \arg \min_{\theta} \mathbb{E}_t \left\{ \lambda(t) \mathbb{E}_{\mathbf{x}(0)} \mathbb{E}_{\mathbf{x}(t)|\mathbf{x}(0)} \left[\left\| \mathbf{s}_{\theta}(\mathbf{x}(t), t) - \nabla_{\mathbf{x}(t)} \log p_{0t}(\mathbf{x}(t) | \mathbf{x}(0)) \right\|_2^2 \right] \right\}$$

loss

Score-Based Through SDE – обобщение

SMLD и DDPM обобщаются до разных стх диффузов

SMLD

$$\mathbf{x}_i = \mathbf{x}_{i-1} + \sqrt{\sigma_i^2 - \sigma_{i-1}^2} \mathbf{z}_{i-1}, \quad i = 1, \dots, N$$

DDPM

$$\mathbf{x}_i = \sqrt{1 - \beta_i} \mathbf{x}_{i-1} + \sqrt{\beta_i} \mathbf{z}_{i-1}, \quad i = 1, \dots, N.$$

$$d\mathbf{x} = \sqrt{\frac{d[\sigma^2(t)]}{dt}} d\mathbf{w}$$

VP

$$d\mathbf{x} = -\frac{1}{2}\beta(t)\mathbf{x} dt + \sqrt{\beta(t)} d\mathbf{w}$$

Score-Based Through SDE

VE

$$d\mathbf{x} = \sqrt{\frac{d[\sigma^2(t)]}{dt}} d\mathbf{w}$$

VP

$$d\mathbf{x} = -\frac{1}{2}\beta(t)\mathbf{x} dt + \sqrt{\beta(t)} d\mathbf{w}$$

sub-VP

new

$$d\mathbf{x} = -\frac{1}{2}\beta(t)\mathbf{x} dt + \sqrt{\beta(t)(1 - e^{-2\int_0^t \beta(s)ds})} d\mathbf{w}.$$

- $p_{0t}(x(t) \mid x(0)) \sim N(\cdot, \cdot)$

- (хорошо вычисляется)

$$\theta^* = \arg \min_{\theta} \mathbb{E}_t \left\{ \lambda(t) \mathbb{E}_{\mathbf{x}(0)} \mathbb{E}_{\mathbf{x}(t) | \mathbf{x}(0)} \left[\left\| \mathbf{s}_{\theta}(\mathbf{x}(t), t) - \nabla_{\mathbf{x}(t)} \log p_{0t}(\mathbf{x}(t) \mid \mathbf{x}(0)) \right\|_2^2 \right] \right\}$$

Score-Based Through SDE

reverse diffusion samplers.

$$d\mathbf{x} = \mathbf{f}(\mathbf{x}, t)dt + \mathbf{G}(t)d\mathbf{w},$$

$$\mathbf{x}_{i+1} = \mathbf{x}_i + \mathbf{f}_i(\mathbf{x}_i) + \mathbf{G}_i\mathbf{z}_i, \quad i = 0, 1, \dots, N-1$$



$$d\mathbf{x} = [\mathbf{f}(\mathbf{x}, t) - \mathbf{G}(t)\mathbf{G}(t)^\top \nabla_{\mathbf{x}} \log p_t(\mathbf{x})]dt + \mathbf{G}(t)d\bar{\mathbf{w}},$$

$$\mathbf{x}_i = \mathbf{x}_{i+1} - \mathbf{f}_{i+1}(\mathbf{x}_{i+1}) + \mathbf{G}_{i+1}\mathbf{G}_{i+1}^\top \mathbf{s}_{\theta^*}(\mathbf{x}_{i+1}, i+1) + \mathbf{G}_{i+1}\mathbf{z}_{i+1},$$

Пере использовали forward дискретизацию в reverse sde

Score-Based Through SDE – predictor-corrector

SMLD

Algorithm 2 PC sampling (VE SDE)

1: $\mathbf{x}_N \sim \mathcal{N}(\mathbf{0}, \sigma_{\max}^2 \mathbf{I})$
2: **for** $i = N - 1$ **to** 0 **do**
3: $\mathbf{x}'_i \leftarrow \mathbf{x}_{i+1} + (\sigma_{i+1}^2 - \sigma_i^2) \mathbf{s}_{\theta^*}(\mathbf{x}_{i+1}, \sigma_{i+1})$
4: $\mathbf{z} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$
5: $\mathbf{x}_i \leftarrow \mathbf{x}'_i + \sqrt{\sigma_{i+1}^2 - \sigma_i^2} \mathbf{z}$
6: **for** $j = 1$ **to** M **do**
7: $\mathbf{z} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$
8: $\mathbf{x}_i \leftarrow \mathbf{x}_i + \epsilon_i \mathbf{s}_{\theta^*}(\mathbf{x}_i, \sigma_i) + \sqrt{2\epsilon_i} \mathbf{z}$
9: **return** $\mathbf{x}_0$

DDPM

Algorithm 3 PC sampling (VP SDE)

1: $\mathbf{x}_N \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$
2: **for** $i = N - 1$ **to** 0 **do**
3: $\mathbf{x}'_i \leftarrow (2 - \sqrt{1 - \beta_{i+1}}) \mathbf{x}_{i+1} + \beta_{i+1} \mathbf{s}_{\theta^*}(\mathbf{x}_{i+1}, i + 1)$
4: $\mathbf{z} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$
5: $\mathbf{x}_i \leftarrow \mathbf{x}'_i + \sqrt{\beta_{i+1}} \mathbf{z}$ Predictor
6: **for** $j = 1$ **to** M **do** Corrector
7: $\mathbf{z} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$
8: $\mathbf{x}_i \leftarrow \mathbf{x}_i + \epsilon_i \mathbf{s}_{\theta^*}(\mathbf{x}_i, i) + \sqrt{2\epsilon_i} \mathbf{z}$
9: **return** $\mathbf{x}_0$

Score-Based Through SDE

		Variance Exploding SDE (SMLD)				Variance Preserving SDE (DDPM)			
FID↓	Sampler	P1000	P2000	C2000	PC1000	P1000	P2000	C2000	PC1000
Predictor									
	ancestral sampling	4.98 ± .06	4.88 ± .06		3.62 ± .03	3.24 ± .02	3.24 ± .02		3.21 ± .02
	reverse diffusion	4.79 ± .07	4.74 ± .08	20.43 ± .07	3.60 ± .02	3.21 ± .02	3.19 ± .02	19.06 ± .06	3.18 ± .01
	probability flow	15.41 ± .15	10.54 ± .08		3.51 ± .04	3.59 ± .04	3.23 ± .03		3.06 ± .03



Score-Based Through SDE - ODE

SDE
$$d\mathbf{x} = \mathbf{f}(\mathbf{x}, t)dt + g(t)d\mathbf{w}$$

rev SDE
$$d\mathbf{x} = [\mathbf{f}(\mathbf{x}, t) - g(t)^2 \nabla_{\mathbf{x}} \log p_t(\mathbf{x})]dt + g(t)d\bar{\mathbf{w}}$$

rev ODE
$$d\mathbf{x} = \left[\mathbf{f}(\mathbf{x}, t) - \frac{1}{2}g(t)^2 \nabla_{\mathbf{x}} \log p_t(\mathbf{x}) \right]dt$$

- Детерминированный
- Те же плотности $p_t(\mathbf{x})$
- Можно вычислить likelihood, вместо vlb
- Детерминировано, биективно $x(0) \rightarrow x(T)$ и $x(T) \rightarrow x(0)$

Score-Based Through SDE - результаты

Table 2: NLLs and FIDs (ODE) on CIFAR-10.

Model	NLL Test ↓	FID ↓
RealNVP (Dinh et al., 2016)	3.49	-
iResNet (Behrmann et al., 2019)	3.45	-
Glow (Kingma & Dhariwal, 2018)	3.35	-
MintNet (Song et al., 2019b)	3.32	-
Residual Flow (Chen et al., 2019)	3.28	46.37
FFJORD (Grathwohl et al., 2018)	3.40	-
Flow++ (Ho et al., 2019)	3.29	-
DDPM (L) (Ho et al., 2020)	$\leq 3.70^*$	13.51
DDPM (L_{simple}) (Ho et al., 2020)	$\leq 3.75^*$	3.17
DDPM	3.28	3.37
DDPM cont. (VP)	3.21	3.69
DDPM cont. (sub-VP)	3.05	3.56
DDPM++ cont. (VP)	3.16	3.93
DDPM++ cont. (sub-VP)	3.02	3.16
DDPM++ cont. (deep, VP)	3.13	3.08
DDPM++ cont. (deep, sub-VP)	2.99	2.92

Table 3: CIFAR-10 sample quality.

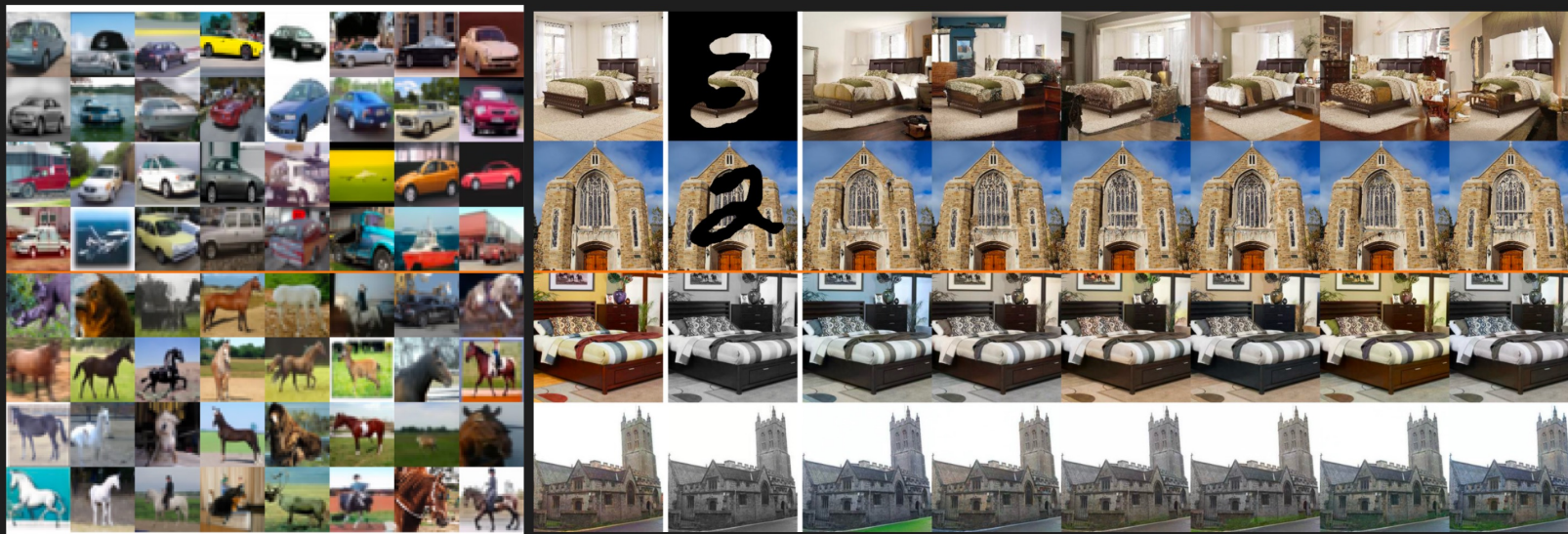
Model	FID↓	IS↑
Conditional		
BigGAN (Brock et al., 2018)	14.73	9.22
StyleGAN2-ADA (Karras et al., 2020a)	2.42	10.14
Unconditional		
StyleGAN2-ADA (Karras et al., 2020a)	2.92	9.83
NCSN (Song & Ermon, 2019)	25.32	$8.87 \pm .12$
NCSNv2 (Song & Ermon, 2020)	10.87	$8.40 \pm .07$
DDPM (Ho et al., 2020)	3.17	$9.46 \pm .11$
DDPM++	2.78	9.64
DDPM++ cont. (VP)	2.55	9.58
DDPM++ cont. (sub-VP)	2.61	9.56
DDPM++ cont. (deep, VP)	2.41	9.68
DDPM++ cont. (deep, sub-VP)	2.41	9.57
NCSN++	2.45	9.73
NCSN++ cont. (VE)	2.38	9.83
NCSN++ cont. (deep, VE)	2.20	9.89

Score-Based Through SDE

$$d\mathbf{x} = \{\mathbf{f}(\mathbf{x}, t) - g(t)^2[\nabla_{\mathbf{x}} \log p_t(\mathbf{x}) + \nabla_{\mathbf{x}} \log p_t(\mathbf{y} \mid \mathbf{x})]\}dt + g(t)d\bar{\mathbf{w}}.$$

y:

- Метка класса
- Чб картинка
- Не полная картинка



Score-Based Through SDE

Привнесли

- Непрерывное обобщение диффузионок
- Новые алгоритмы семплинга
- Подсчет likelihood
- Детерминированное биективное отображение данных в шум
- Условная генерация
- Топ модели по NLL, FID, IS

А все-таки

